Transaction in SQL Server (Detailed Explanation)

In **SQL Server**, a **Transaction** is a logical unit of work that consists of one or more SQL statements executed as a single operation.
The main goal of a transaction is to **ensure data integrity and consistency** in the database.

If all operations inside the transaction succeed → the changes are **permanently saved**.
If any operation fails → the entire transaction can be **rolled back** (undone).

In simple terms:

Transaction = Group of SQL statements executed together as one unit.

Example scenario:

A **bank transfer** from Account A to Account B.

Steps involved:

1. Deduct money from Account A
2. Add money to Account B

Both steps must succeed.
If one step fails, the database should not keep partial results.

---

1. Basic Transaction Syntax in SQL Server

SQL Server provides three main commands:

- BEGIN TRANSACTION
- COMMIT
- ROLLBACK

Syntax

```
BEGIN TRANSACTION;


-- SQL statements


COMMIT;
```

or

```
BEGIN TRANSACTION;

-- SQL statements

ROLLBACK;
```

---

2. Example of Transaction

Assume we have a banking table.

```
CREATE TABLE Accounts
(
   AccountID INT PRIMARY KEY,
   CustomerName VARCHAR(50),
   Balance MONEY
);
```

Sample Data:

AccountID	CustomerName	Balance
101	Ravi	10000
102	Anita	5000

Money Transfer Example

Transfer **2000 from Ravi to Anita**

```
BEGIN TRANSACTION;

UPDATE Accounts
```

```
SET Balance = Balance - 2000
WHERE AccountID = 101;

UPDATE Accounts
SET Balance = Balance + 2000
WHERE AccountID = 102;

COMMIT;
```

Result:

Account	Balance
101	8000
102	7000

Both operations succeed → **transaction committed**.

---

3. Rollback Transaction Example

Suppose second update fails.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 2000
WHERE AccountID = 101;

-- error occurs here
UPDATE Accounts
SET Balance = Balance + 2000
```

```
WHERE AccountID = 999;


ROLLBACK;
```

Result:

No changes will be saved.

Account	Balance
101	10000
102	5000

Because we executed **ROLLBACK**.

---

4. Transaction Control Commands

1️⃣ BEGIN TRANSACTION

Starts a transaction.

```
BEGIN TRANSACTION;
```

or

```
BEGIN TRAN;
```

2️⃣ COMMIT

Saves all changes permanently.

```
COMMIT;
```

Once committed:

- Changes cannot be undone.

3️⃣ ROLLBACK

Cancels all changes made in the transaction.

ROLLBACK;

Database returns to previous state.

---

5. ACID Properties of Transactions

Transactions follow **ACID principles**.

A — Atomicity

Atomic means **all or nothing**.

Either:

- All operations succeed
- Or none are applied.

Example:

Debit ₹1000 from A but credit fails → system must rollback.

C — Consistency

Database moves from **one valid state to another valid state**.

Example:

Account balance cannot become negative if constraint exists.

I — Isolation

Transactions are isolated from each other.

Multiple users executing transactions simultaneously **should not interfere**.

Example:
Two users updating the same account simultaneously.

SQL Server manages this using **locks**.

D — Durability

Once transaction is committed, data is **permanently stored**.

Even if:

- Server crashes
- Power failure occurs

SQL Server uses **Transaction Logs** to recover.

---

6. Transaction Log

SQL Server maintains a **Transaction Log file (.ldf)**.

It records:

- All transactions
- All changes made to the database

Example recorded operations:

Insert

Update

Delete

Commit

Rollback

Purpose:

1. Recovery after crash
2. Rollback transactions
3. Maintain data consistency

---

7. Using TRY...CATCH with Transactions

Transactions are commonly used with **exception handling**.

Example:

BEGIN TRY

```
    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - 2000
    WHERE AccountID = 101;

    UPDATE Accounts
    SET Balance = Balance + 2000
    WHERE AccountID = 102;

    COMMIT;

END TRY

BEGIN CATCH

    ROLLBACK;

    PRINT 'Transaction Failed';

END CATCH
```

If error occurs → transaction automatically rolled back.

---

10. Savepoints (Partial Rollback)

Savepoints allow rolling back **part of a transaction**.

Example:

```
BEGIN TRANSACTION;

INSERT INTO Accounts VALUES (105,'Arun',2000);

SAVE TRANSACTION SavePoint1;

INSERT INTO Accounts VALUES (106,'Kiran',3000);

ROLLBACK TRANSACTION SavePoint1;

COMMIT;
```

Result:

Only first insert is saved.

AccountID	Customer
105	Arun

---

11. Types of Transactions in SQL Server

1️⃣ Auto Commit Transaction

Default mode.

Each SQL statement automatically commits.

Example:

```
INSERT INTO Accounts VALUES (107,'Raj',5000);
```

SQL Server automatically commits.

2️⃣ Explicit Transaction

User manually controls transaction.

Example:

```
BEGIN TRANSACTION

UPDATE Accounts

SET Balance = Balance - 1000

WHERE AccountID = 101


COMMIT
```

---

12. Real World Use Cases of Transactions

Transactions are used in:

Banking Systems

Money transfer between accounts.

E-Commerce

Order creation

Insert Order

Insert Order Items

Update Inventory

Process Payment

Payroll Systems

Salary calculations.

Airline Booking

Seat reservation.

---

13. Common Interview Questions

1. What is a transaction in SQL Server?

A transaction is a group of SQL statements executed as a single logical unit of work.

---

2. What are transaction control statements?

- BEGIN TRANSACTION
- COMMIT
- ROLLBACK

---

3. What are ACID properties?

- Atomicity
- Consistency
- Isolation
- Durability

---

4. What is a transaction log?

A file that records all database modifications for recovery and rollback.

✅ **Short Summary**

Transaction ensures:

- Data integrity
- Data consistency
- Error recovery
- Safe concurrent access

Core commands:

BEGIN TRANSACTION

COMMIT

ROLLBACK

SAVEPOINT